

Para montar esta arquitectura de alta concurrencia orientada a simulación financiera en **Docker Desktop**, utilizaremos **Docker Compose**. He diseñado la configuración optimizando los recursos de hardware solicitados, asignando límites exactos de CPU y Memoria (RAM) mediante las directivas de control de contenedores de Docker.

He generado un entorno base funcional con **Python 3.12, RabbitMQ y Locust**. Aquí tienes los archivos de configuración listos para producción.

Estructura del Proyecto

Crea la siguiente jerarquía de archivos en tu computadora:

```
|— docker-compose.yml
|— injector/
|   |— Dockerfile
|   |— locustfile.py
|— matching_engine/
|   |— Dockerfile
|   |— engine.py
```



Archivos de Configuración

1. docker-compose.yml (Raíz)

Este archivo define la red y los **límites estrictos de hardware** para simular los tres nodos físicos independientes en tu entorno de Docker Desktop.

```
version: '3.8'

services:
  # Nodo 2: Queue Broker (RabbitMQ)
  queue-broker:
    image: rabbitmq:3-management
    container_name: queue-broker
    ports:
      - "5672:5672" # Puerto de comunicación de datos AMQP
      - "15672:15672" # Interfaz Web de administración
    deploy:
      resources:
        limits:
```

```

        cpus: '2.0'      # Límite: 2 vCPU
        memory: 4096M    # Límite: 4GB RAM
    healthcheck:
        test: ["CMD", "rabbitmq-diagnostics", "check_running"]
        interval: 10s
        timeout: 5s
        retries: 5

```

Nodo 3: Matching Engine Node (Prototipo en RAM con Python 3.12)

```

matching-engine:
    build: ./matching_engine
    container_name: matching-engine
    depends_on:
        queue-broker:
            condition: service_healthy
    deploy:
        resources:
            limits:
                cpus: '4.0'      # Límite: 4 vCPU
                memory: 8192M    # Límite: 8GB RAM
    environment:
        - RABBITMQ_HOST=queue-broker

```

Nodo 1: Inyector de Carga (Locust)

```

injector:
    build: ./injector
    container_name: injector
    ports:
        - "8089:8089"    # Interfaz Web de Locust
    depends_on:
        queue-broker:
            condition: service_healthy
    deploy:
        resources:
            limits:
                cpus: '2.0'      # Límite: 2 vCPU
                memory: 4096M    # Límite: 4GB RAM
    environment:
        - RABBITMQ_HOST=queue-broker

```

2. Configuración del Nodo 3: matching_engine/

-
- **matching_engine/Dockerfile:**

```

FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir pika
COPY engine.py .

```

```
CMD ["python", "engine.py"]
```

- **matching_engine/engine.py:** Utiliza heapq para el procesamiento ultrarrápido en memoria de las estructuras del libro de órdenes (*Order Book*).

```
import pika
import json
import heapq
import os

host = os.getenv('RABBITMQ_HOST', 'localhost')
connection =
pika.BlockingConnection(pika.ConnectionParameters(host=host))
channel = connection.channel()
channel.queue_declare(queue='orders', durable=True)

# Heaps optimizados en RAM para simular el emparejamiento bursátil
buy_orders = [] # Max-heap (precios altos primero -> almacenado
                 como -precio)
sell_orders = [] # Min-heap (precios bajos primero)

print("[-] Matching Engine activo en RAM. Esperando flujo...")

def callback(ch, method, properties, body):
    order = json.loads(body)
    price = float(order['price'])
    side = order['side']

    if side == 'BUY':
        heapq.heappush(buy_orders, (-price, order))
    else:
        heapq.heappush(sell_orders, (price, order))

    print(f"[+] Orden: {order['order_id']} | Lado: {side} | Libros
-> BUYS: {len(buy_orders)} SELLS: {len(sell_orders)}")
    ch.basic_ack(delivery_tag=method.delivery_tag)

channel.basic_consume(queue='orders',
on_message_callback=callback)
channel.start_consuming()
```

-

3. Configuración del Nodo 1: injector/

-

- **injector/Dockerfile:**

```
FROM python:3.12-slim
WORKDIR /app
```

```

RUN pip install --no-cache-dir locust pika
COPY locustfile.py .
CMD ["locust", "-f", "locustfile.py", "--headless", "-u", "10",
"-r", "2", "--run-time", "5m"]

```

- **injector/locustfile.py:** Genera dinámicamente ráfagas del dataset simulado con el esquema exacto de campos requerido.

```

from locust import User, task, between
import pika
import json
import uuid
import random
import time
import os

class RabbitMQUser(User):
    wait_time = between(0.001, 0.01) # Simulación de ráfaga de
alta velocidad

    def on_start(self):
        host = os.getenv('RABBITMQ_HOST', 'localhost')
        self.connection =
pika.BlockingConnection(pika.ConnectionParameters(host=host))
        self.channel = self.connection.channel()
        self.channel.queue_declare(queue='orders', durable=True)
        self.symbols = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'TSLA']

    @task
    def send_order(self):
        # Dataset Simulado Estructurado (Meta: 200,000 registros
de prueba)
        order = {
            "order_id": str(uuid.uuid4()),
            "trader_id": f"TRADER_{random.randint(1, 1000)}",
            "symbol": random.choice(self.symbols),
            "side": random.choice(['BUY', 'SELL']),
            "price": round(random.uniform(10.0, 500.0), 2),
            "quantity": random.randint(1, 500),
            "timestamp": int(time.time() * 1000)
        }

        self.channel.basic_publish(
            exchange='',
            routing_key='orders',
            body=json.dumps(order),
            properties=pika.BasicProperties(delivery_mode=2) #
Mensajes persistentes
        )

```

```
def on_stop(self):  
    self.connection.close()
```

Instrucciones de Despliegue

1. Configurar Docker Desktop:

- Abre la configuración de Docker Desktop (*Settings > Resources*).
- Asegúrate de que los límites globales asignados a la máquina virtual de Docker sean mayores a la suma de tus contenedores (**mínimo 9 vCPUs y 17GB de RAM asignados a la app global** para evitar errores de falta de memoria).

2. Lanzar la infraestructura: En la raíz de tu proyecto ejecuta:

```
docker-compose up --build
```

3. Monitorear:

- **RabbitMQ Dashboard:** Accede a <http://localhost:15672> (Usuario/Contraseña por defecto: guest / guest) para visualizar el tráfico y saturación de la cola en el Nodo 2.
- **Consola del Matching Engine:** Inspecciona los logs de matching-engine para observar la manipulación de estructuras heapq en tiempo real.

Si lo deseas, indícame:

-
- ¿Prefieres cambiar la carga automatizada por la lectura de un **archivo físico CSV/JSON estático de 200k filas** en lugar de generarlo de forma aleatoria?
- ¿Necesitas la lógica avanzada de negocio para la **función de cruce automático (match)** en engine.py cuando coincidan los precios de compra y venta?
-